

International Cybersecurity and Digital Forensics Academy (ICDFA)

Basic Computer Skills for Digital Forensics

Firewall Traffic Control and Forensic Verification

Student Name: Kafayat Omolara Animashawun

Student ID: 2025/FWSD/11317

Instructor Name: Aminu Idris

Date of Submission: 25/09/2026

EXECUTIVE SUMMARY

This practical exercise focused on the forensic investigation of DNS spoofing within a controlled and isolated laboratory environment. The exercise was conducted using a Kali Linux virtual machine together with dedicated Linux network namespaces representing the victim, legitimate DNS endpoint, and controlled spoofing endpoint.

The investigation began by documenting the baseline network configuration, including network interfaces, IP addressing, routing information, ARP/neighbor relationships, DNS configuration, and active hosts within the laboratory network. A harmless training webpage was also created and deployed through Apache to provide a controlled web resource for examining the relationship between DNS resolution and subsequent HTTP activity.

A dedicated isolated laboratory network using the 10.203.0.0/24 address range was subsequently established. The victim was assigned 10.203.0.2, the legitimate DNS endpoint was assigned 10.203.0.1, and the controlled spoofing endpoint was assigned 10.203.0.3. Connectivity and ARP relationships between the laboratory endpoints were verified before introducing the DNS-spoofing component.

DNS traffic was captured to establish the legitimate communication pattern. The baseline evidence confirmed that queries from the victim for bank.test were directed to the

legitimate DNS endpoint and that the expected response resolved the domain to 10.203.0.1. Packet captures and SHA-256 hashes were preserved as forensic evidence.

The exercise demonstrates the importance of establishing a reliable network baseline before analysing suspected DNS manipulation. Comparing DNS responses with Ethernet source information, ARP mappings, and subsequent HTTP activity provides a structured method for identifying abnormal DNS behaviour and distinguishing it from legitimate network variation.

PURPOSE OF THE ASSIGNMENT

The purpose of this assignment is to provide practical experience in identifying, analysing, and documenting DNS spoofing activity from a digital forensics perspective. The exercise is designed to demonstrate how DNS resolution, ARP communication, Layer 2 identities, and subsequent HTTP connections can be correlated to investigate a suspected DNS manipulation event.

The practical also provides an opportunity to establish a documented network baseline before introducing controlled anomalous activity. By preserving packet captures, network configuration information, ARP observations, scripts, and cryptographic hashes, the exercise demonstrates the importance of maintaining reliable and verifiable evidence throughout a forensic investigation.

All activities were performed within an isolated and controlled laboratory environment using designated test systems and the controlled domain bank.test.

OBJECTIVES OF THE ASSIGNMENT

The objectives of the assignment are to:

- Establish and document the normal network, routing, ARP, and DNS configuration before the controlled simulation.
- Identify the systems and network interfaces participating in the laboratory environment.
- Establish the legitimate DNS resolution behaviour for the controlled domain bank.test.

- Capture and preserve DNS and ARP traffic for forensic analysis.
- Configure a controlled spoofing endpoint within an isolated Linux network namespace.
- Examine the relationship between DNS responses and the underlying Ethernet MAC addresses.
- Identify abnormal or conflicting DNS responses and determine the associated responder.
- Correlate DNS resolution activity with subsequent HTTP communication.
- Preserve relevant packet captures, command outputs, scripts, and cryptographic hashes as forensic evidence.
- Distinguish controlled anomalous DNS behaviour from legitimate network variation.
- Document appropriate cleanup and defensive considerations following the laboratory exercise.

TOOLS AND RESOURCES USED

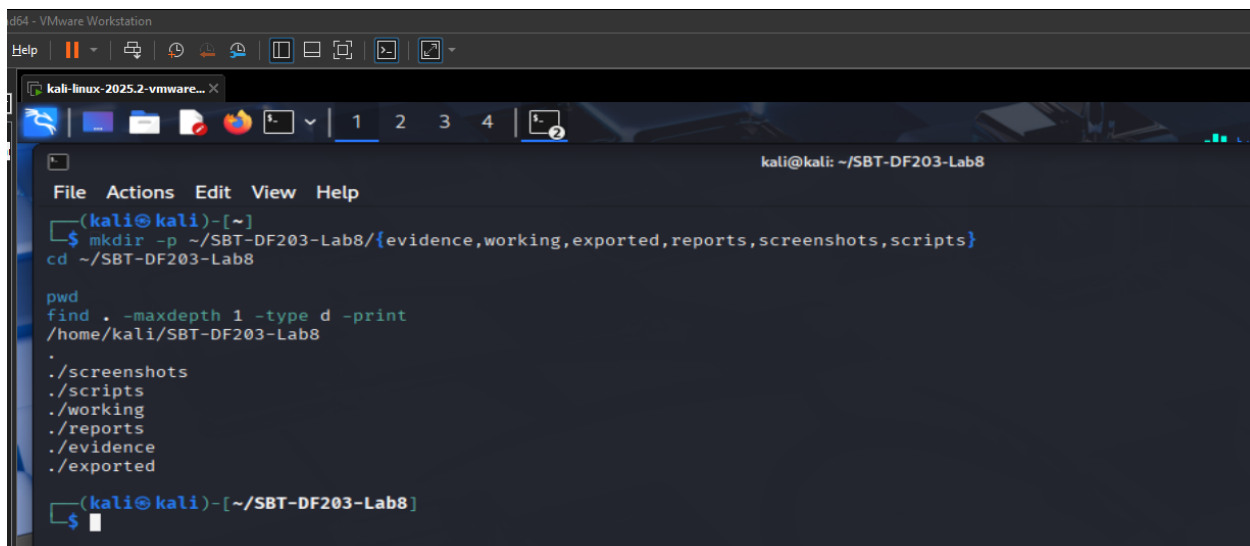
Category	Tool / Resource	Purpose / Usage
Operating System	Kali Linux	Primary forensic analysis and laboratory environment
Virtualisation / Networking	Linux Network Namespaces	Used to create isolated victim and controlled spoofing endpoints
Virtual Networking	labbr Linux Bridge	Provided the isolated Layer 2 laboratory network
Network Configuration	ip	Interface, IP addressing, routing, namespace, and bridge configuration
ARP Analysis	ip neigh	Inspection of ARP/neighbor cache entries
Host Discovery	arp-scan	Layer 2 host discovery and MAC address identification
Host Discovery	nmap	Network host discovery and service identification

Category	Tool / Resource	Purpose / Usage
Packet Capture	tcpdump	Capturing and reviewing DNS, ARP, and network traffic
Packet Analysis	tshark	Command-line analysis of captured packets
Packet Analysis	Wireshark	Graphical inspection and analysis of packet captures
Network Services	ss	Verification of listening network services and ports
Firewall	iptables	Controlled firewall traffic management within the laboratory
HTTP Testing	curl	Verification of the Apache-hosted training webpage
File Integrity	sha256sum	Generation of SHA-256 hashes for evidence integrity verification
Scripting	Python	Development of the controlled DNS responder scripts
DNS Analysis	dnslib	Parsing and constructing DNS requests and responses
Packet Crafting	Scapy	Controlled packet generation and network analysis
Web Server	Apache HTTP Server	Hosting the harmless training webpage used during the exercise
Test Domain	bank.test	Controlled domain used for DNS resolution and spoofing analysis
Legitimate DNS Endpoint	10.203.0.1	Provided the legitimate DNS response in the laboratory

Category	Tool / Resource	Purpose / Usage
Victim Endpoint	10.203.0.2	Represented the client making DNS queries
Controlled Spoofing Endpoint	10.203.0.3	Represented the controlled DNS-spoofing responder
Evidence Storage	evidence/, reports/, scripts/, working/	Organised packet captures, reports, scripts, and supporting laboratory evidence

Lab Environment and Network Configuration

The SBT-DF203 Lab 8 practical was conducted from a Kali Linux virtual machine prepared for the DNS spoofing forensic exercise. A dedicated laboratory workspace named SBT-DF203-Lab8 was created to maintain an organized separation of evidence, working files, exported analysis results, reports, screenshots and scripts. The workspace was verified before commencing the practical to ensure that evidence generated during the exercise could be systematically preserved and referenced during the final forensic report.



```

kali@kali: ~/SBT-DF203-Lab8
File Actions Edit View Help
(kali@kali)~$ mkdir -p ~/SBT-DF203-Lab8/{evidence,working,exported,reports,screenshots,scripts}
cd ~/SBT-DF203-Lab8

pwd
/home/kali/SBT-DF203-Lab8
find . -maxdepth 1 -type d -print
./screenshots
./scripts
./working
./reports
./evidence
./exported

(kali@kali)~[~/SBT-DF203-Lab8]
$

```

Initial Network Configuration

The network configuration of the Kali analyst virtual machine was documented before

commencing the controlled DNS spoofing exercise. The primary network interface was identified as eth0, with the system assigned IPv4 address 192.168.232.128/24. The host was connected to the 192.168.232.0/24 network, providing the initial network context required for identifying the victim, gateway and DNS resolver during the subsequent forensic investigation.

```
(kali㉿kali)-[~/SBT-DF203-Lab8]
$ ip -br address
lo                UNKNOWN      127.0.0.1/8 ::1/128
eth0              UP          192.168.232.128/24 fe80::d5cc:fb25:aa2d:b901/64
docker0          DOWN        172.17.0.1/16
br-d2696585dc9a  UP          172.18.0.1/16 fe80::8069:abff:fe01:9c62/64
br-ec90d8c3abfb  DOWN        172.19.0.1/16
vethc45370c@if2  UP          fe80::10cf:85ff:fe44:ec54/64
veth9e191ba@if2  UP          fe80::b00b:88ff:fed1:a7f1/64

(kali㉿kali)-[~/SBT-DF203-Lab8]
$ ip route
default via 192.168.232.2 dev eth0 proto dhcp src 192.168.232.128 metric 100
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
172.18.0.0/16 dev br-d2696585dc9a proto kernel scope link src 172.18.0.1
172.19.0.0/16 dev br-ec90d8c3abfb proto kernel scope link src 172.19.0.1 linkdown
192.168.232.0/24 dev eth0 proto kernel scope link src 192.168.232.128 metric 100
```

Routing Configuration

The routing table of the Kali analyst virtual machine was recorded as part of the pre-simulation baseline. The system used 192.168.232.2 as its default gateway through interface eth0, while the directly connected 192.168.232.0/24 network was also associated with the same interface. The routing information was preserved before any controlled interception activity to provide a reference for interpreting subsequent network traffic and determining the communication path used during the exercise.

```
(kali㉿kali)-[~/SBT-DF203-Lab8]
$ ip route
default via 192.168.232.2 dev eth0 proto dhcp src 192.168.232.128 metric 100
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
172.18.0.0/16 dev br-d2696585dc9a proto kernel scope link src 172.18.0.1
172.19.0.0/16 dev br-ec90d8c3abfb proto kernel scope link src 172.19.0.1 linkdown
192.168.232.0/24 dev eth0 proto kernel scope link src 192.168.232.128 metric 100
```

Initial ARP State

The initial ARP neighbor state was recorded before the DNS spoofing simulation. At this stage, the Kali analyst VM had an established neighbor entry for the gateway 192.168.232.2, which resolved to MAC address 00:50:56:fe:76:b0 and was reported as reachable through eth0. This information establishes the legitimate IP-to-MAC

relationship that will serve as a baseline when analysing ARP activity during the controlled man-in-the-middle and DNS spoofing stages.

```
(kali@kali)-[~/SBT-DF203-Lab8]
$ ip -br address | tee reports/interfaces.txt
ip route | tee reports/routes.txt
ip neigh show | tee reports/arp_before.txt
lo                UNKNOWN          127.0.0.1/8 ::1/128
eth0              UP                192.168.232.128/24 fe80::d5cc:fb25:aa2d:b901/64
docker0          DOWN            172.17.0.1/16
br-d2696585dc9a  UP                172.18.0.1/16 fe80::8069:abff:fe01:9c62/64
br-ec90d8c3abfb  DOWN            172.19.0.1/16
vethc45370c@if2  UP                fe80::10cf:85ff:fe44:ec54/64
vethb1059fb@if2  UP                fe80::a4f6:e6ff:fe09:4eeb/64
default via 192.168.232.2 dev eth0 proto dhcp src 192.168.232.128 metric 100
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
172.18.0.0/16 dev br-d2696585dc9a proto kernel scope link src 172.18.0.1
172.19.0.0/16 dev br-ec90d8c3abfb proto kernel scope link src 172.19.0.1 linkdown
192.168.232.0/24 dev eth0 proto kernel scope link src 192.168.232.128 metric 100
192.168.232.2 dev eth0 lladdr 00:50:56:fe:76:b0 REACHABLE
```

Harmless Training Page Creation

A dedicated HTML training page was created within the Lab 8 working directory to provide a controlled and harmless web resource for the DNS spoofing forensic simulation. The page was specifically designed for the authorized laboratory exercise and contains only identification and training information. It does not contain login functionality, credential fields, or mechanisms for collecting personal information.

```
(kali@kali)-[~/SBT-DF203-Lab8]
$ cat > working/index.html <<'EOF'
<!DOCTYPE html>
<html>
<head>
  <title>ICDFA DNS Spoofing Training</title>
</head>
<body style="font-family: Arial, sans-serif;">
  <h1>ICDFA DNS Spoofing Training Page</h1>
  <p>This is an authorized cybersecurity training simulation.</p>
  <p>No credentials or personal information are collected.</p>
  <p>DNS Spoofing Forensics – SBT-DF203 Lab 8</p>
</body>
</html>
EOF
```

Training Page Content Verification

After creating the HTML file, its contents were displayed from the terminal using cat working/index.html. This verification confirmed that the file contained the expected training-page content before it was deployed to the local Apache web server. Reviewing the file at this stage provided an initial confirmation that the controlled web resource was correctly prepared and that no unintended content had been introduced.

```
(kali@kali)-[~/SBT-DF203-Lab8]
$ cat working/index.html
<!DOCTYPE html>
<html>
<head>
  <title>ICDFA DNS Spoofing Training</title>
</head>
<body style="font-family: Arial, sans-serif;">
  <h1>ICDFA DNS Spoofing Training Page</h1>
  <p>This is an authorized cybersecurity training simulation.</p>
  <p>No credentials or personal information are collected.</p>
  <p>DNS Spoofing Forensics – SBT-DF203 Lab 8</p>
</body>
</html>
```

Deployment to Apache Web Server

The verified training page was copied from the Lab 8 working directory into the Apache web server document root at /var/www/html/index.html. This step made the controlled page available through the local web server and established the web content that could subsequently be associated with the replacement destination during the DNS spoofing simulation. The copy operation was performed twice during preparation; the repeated operation did not alter the page content.

```
(kali@kali)-[~/SBT-DF203-Lab8]
$ sudo cp working/index.html /var/www/html/index.html

(kali@kali)-[~/SBT-DF203-Lab8]
$ sudo cp working/index.html /var/www/html/index.html

(kali@kali)-[~/SBT-DF203-Lab8]
$ ls -l /var/www/html/index.html
-rw-r--r-- 1 root root 381 Sep 22 13:37 /var/www/html/index.html

(kali@kali)-[~/SBT-DF203-Lab8]
$ sha256sum /var/www/html/index.html | tee reports/training_page_sha256.txt
2fcc03722008fe1b158840e6cb503faafe1bf148f8a585b7819872b133bc89e /var/www/html/index.html
```

Deployed File Verification

The deployed HTML file was then verified using `ls -l /var/www/html/index.html`. The resulting output confirmed that the file existed in the Apache document root, had a file size of 381 bytes, and was owned by the root account with read permissions available to other users. This verification established that the intended training resource had been successfully placed in the web server's document root before proceeding with further laboratory activities.

```
(kali㉿kali)-[~/SBT-DF203-Lab8]
$ sudo cp working/index.html /var/www/html/index.html

(kali㉿kali)-[~/SBT-DF203-Lab8]
$ sudo cp working/index.html /var/www/html/index.html

(kali㉿kali)-[~/SBT-DF203-Lab8]
$ ls -l /var/www/html/index.html
-rw-r--r-- 1 root root 381 Sep 22 13:37 /var/www/html/index.html

(kali㉿kali)-[~/SBT-DF203-Lab8]
$ sha256sum /var/www/html/index.html | tee reports/training_page_sha256.txt
2fcc03722008fe1b158840e6cb503faafe1bf148f8a585b7819872b133bc89e /var/www/html/index.html
```

SHA-256 Integrity Baseline

A SHA-256 cryptographic hash was generated for the deployed training page and saved to reports/training_page_sha256.txt. The resulting value, 2fcc03722008fe1b158840e6cb503faafe1bf148f8a585b7819872b133bc89e, establishes an integrity baseline for the exact web content used during the exercise. This hash can be referenced later to determine whether the controlled training page was modified during the DNS spoofing forensic investigation.

```
(kali㉿kali)-[~/SBT-DF203-Lab8]
$ sudo cp working/index.html /var/www/html/index.html

(kali㉿kali)-[~/SBT-DF203-Lab8]
$ sudo cp working/index.html /var/www/html/index.html

(kali㉿kali)-[~/SBT-DF203-Lab8]
$ ls -l /var/www/html/index.html
-rw-r--r-- 1 root root 381 Sep 22 13:37 /var/www/html/index.html

(kali㉿kali)-[~/SBT-DF203-Lab8]
$ sha256sum /var/www/html/index.html | tee reports/training_page_sha256.txt
2fcc03722008fe1b158840e6cb503faafe1bf148f8a585b7819872b133bc89e /var/www/html/index.html
```

Apache Web Server Service Verification

Following deployment of the harmless training page, the Apache HTTP Server service was checked to confirm that the local web server was operational before proceeding with the DNS spoofing simulation. The service status showed that apache2.service was loaded and active (running), with the Apache worker processes successfully initialized. The service had started at approximately 13:27 on 22 September 2026, confirming that the web server was available to provide the controlled training content during the laboratory exercise.

```

(kali@kali)-[~/SBT-DF203-Lab8]
$ sudo systemctl status apache2 --no-pager
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: disabled)
   Active: active (running) since Tue 2026-09-22 13:27:46 EDT; 15min ago
     Invocation: af81951a53504da1a8d4a0a019f00fd2
       Docs: https://httpd.apache.org/docs/2.4/
    Main PID: 27527 (apache2)
   Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served/sec: 0 B/sec"
     Tasks: 6 (limit: 8113)
    Memory: 40.1M (peak: 40.3M)
       CPU: 991ms
    CGroup: /system.slice/apache2.service
            └─27527 /usr/sbin/apache2 -k start -DFOREGROUND
              └─27540 /usr/sbin/apache2 -k start -DFOREGROUND
                └─27541 /usr/sbin/apache2 -k start -DFOREGROUND
                  └─27542 /usr/sbin/apache2 -k start -DFOREGROUND
                    └─27543 /usr/sbin/apache2 -k start -DFOREGROUND
                      └─27544 /usr/sbin/apache2 -k start -DFOREGROUND

Sep 22 13:27:45 kali systemd[1]: Starting apache2.service - The Apache HTTP Server...
Sep 22 13:27:45 kali apachectl[27527]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1...his message
Sep 22 13:27:46 kali systemd[1]: Started apache2.service - The Apache HTTP Server.
Hint: Some lines were ellipsized, use -l to show in full.

```

Training Page HTTP Verification

The availability of the deployed training page was subsequently verified by issuing an HTTP request to the local Apache server using `curl http://127.0.0.1/`. The server returned the expected HTML content, including the title “ICDFA DNS Spoofing Training”, the statement confirming that the activity was an authorized cybersecurity training simulation, and the explicit notice that no credentials or personal information were collected. This confirmed that the intended training page was being served correctly by Apache and established the local web service as ready for the subsequent controlled DNS spoofing investigation.

```

(kali@kali)-[~/SBT-DF203-Lab8]
$ curl http://127.0.0.1/
<!DOCTYPE html>
<html>
<head>
  <title>ICDFA DNS Spoofing Training</title>
</head>
<body style="font-family: Arial, sans-serif;">
  <h1>ICDFA DNS Spoofing Training Page</h1>
  <p>This is an authorized cybersecurity training simulation.</p>
  <p>No credentials or personal information are collected.</p>
  <p>DNS Spoofing Forensics - SBT-DF203 Lab 8</p>
</body>
</html>

```

DNS Resolver Configuration

The DNS resolver configuration of the Kali analyst VM was examined before commencing the controlled DNS spoofing exercise. The `/etc/resolv.conf` file identified `127.0.0.53` as the configured nameserver, indicating that local DNS requests are handled through the `systemd-resolved` stub resolver. The loopback address represents the local DNS service rather than an external DNS server. This distinction was recorded to provide appropriate context when interpreting subsequent DNS traffic captured from the network interface.

```

(kali@kali)-[~/SBT-DF203-Lab8]
$ cat /etc/resolv.conf
# This is /run/systemd/resolve/stub-resolv.conf managed by man:systemd-resolved(8).
# Do not edit.
#
# This file might be symlinked as /etc/resolv.conf. If you're looking at
# /etc/resolv.conf and seeing this text, you have followed the symlink.
#
# This is a dynamic resolv.conf file for connecting local clients to the
# internal DNS stub resolver of systemd-resolved. This file lists all
# configured search domains.
#
# Run "resolvectl status" to see details about the uplink DNS servers
# currently in use.
#
# Third party programs should typically not access this file directly, but only
# through the symlink at /etc/resolv.conf. To manage man:resolv.conf(5) in a
# different way, replace this symlink by a static file or a different symlink.
#
# See man:systemd-resolved.service(8) for details about the supported modes of
# operation for /etc/resolv.conf.

nameserver 127.0.0.53
options edns0 trust-ad
search .

```

Upstream DNS Server Identification

The active upstream DNS configuration was subsequently identified using `resolvectl`. The `eth0` interface was configured as the default DNS route, with `1.1.1.1` identified as the currently selected DNS server and `8.8.8.8` configured as an additional DNS server. These observations establish the normal DNS configuration of the analyst VM before the controlled simulation. The configuration was recorded separately from the victim's DNS settings because the analyst and victim systems may use different DNS configurations.

```

(kali@kali)-[~/SBT-DF203-Lab8]
$ resolvectl status 2>/dev/null | sed -n '1,120p'
Global
  Protocols: +LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
  resolv.conf mode: stub

Link 4 (br-d2696585dc9a)
  Current Scopes: LLMNR/IPv4 LLMNR/IPv6
  Protocols: -DefaultRoute +LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
  Default Route: no

Link 195 (veth0a59e9a)
  Current Scopes: LLMNR/IPv6
  Protocols: -DefaultRoute +LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
  Default Route: no

Link 6 (vethc45370c)
  Current Scopes: LLMNR/IPv6
  Protocols: -DefaultRoute +LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
  Default Route: no

Link 5 (br-ec90d8c3abfb)
  Current Scopes: none
  Protocols: -DefaultRoute +LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
  Default Route: no

Link 2 (eth0)
  Current Scopes: DNS LLMNR/IPv4 LLMNR/IPv6
  Protocols: +DefaultRoute +LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
  Current DNS Server: 1.1.1.1
  DNS Servers: 1.1.1.1 8.8.8.8
  Default Route: yes

Link 3 (docker0)
  Current Scopes: none
  Protocols: -DefaultRoute +LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
  Default Route: no

```

Parameter	Observation
Interface	eth0
DNS scope	DNS enabled
Current DNS server	1.1.1.1
Additional DNS server	8.8.8.8
Default DNS route	Yes

ARP Network Discovery

An ARP scan was performed on the active eth0 interface to establish the Layer 2 hosts visible within the authorised virtual laboratory network. Three IP addresses responded to ARP requests: 192.168.232.1, 192.168.232.2 and 192.168.232.254. Their corresponding MAC addresses were recorded, with all three addresses identified by the scanning tool as VMware-associated interfaces. The analyst VM was identified as 192.168.232.128 with MAC address 00:0c:29:77:75:3c. The scan establishes the initial IP-to-MAC relationships but does not, by itself, establish the functional role of each discovered host.

```
(kali@kali)-[~/SBT-DF203-Lab8]
$ sudo arp-scan --interface=eth0 --localnet
Interface: eth0, type: EN10MB, MAC: 00:0c:29:77:75:3c, IPv4: 192.168.232.128
Starting arp-scan 1.10.0 with 256 hosts (https://github.com/royhills/arp-scan)
192.168.232.1    00:50:56:c0:00:08    VMware, Inc.
192.168.232.2    00:50:56:fe:76:b0    VMware, Inc.
192.168.232.254 00:50:56:ed:bd:0b    VMware, Inc.

4 packets received by filter, 0 packets dropped by kernel
Ending arp-scan 1.10.0: 256 hosts scanned in 2.282 seconds (112.18 hosts/sec). 3 responded
```

IP Address	MAC Address	Vendor
192.168.232.1	00:50:56:c0:00:08	VMware, Inc.
192.168.232.2	00:50:56:fe:76:b0	VMware, Inc.
192.168.232.254	00:50:56:ed:bd:0b	VMware, Inc.

Analyst ARP/Neighbour Cache

The analyst VM's neighbour table was examined and preserved as part of the pre-event baseline. The table recorded an IP-to-MAC association for 192.168.232.2 at 00:50:56:fe:76:b0, corresponding to the previously identified default gateway, and an

additional association for 192.168.232.254 at 00:50:56:ed:bd:0b. The neighbour-state values were also recorded. Because Linux neighbour states such as REACHABLE and STALE represent cache-validation states rather than security classifications, they were not treated as indicators of malicious activity.

```
(kali@kali)~[~/SBT-DF203-Lab8]
$ ip neigh show dev eth0 | tee reports/arp_baseline.txt
192.168.232.254 lladdr 00:50:56:ed:bd:0b STALE
192.168.232.2 lladdr 00:50:56:fe:76:b0 REACHABLE

(kali@kali)~[~/SBT-DF203-Lab8]
$ sudo arp-scan --interface=eth0 --localnet | tee reports/arp_scan_baseline.txt
Interface: eth0, type: EN10MB, MAC: 00:0c:29:77:75:3c, IPv4: 192.168.232.128
Starting arp-scan 1.10.0 with 256 hosts (https://github.com/royhills/arp-scan)
192.168.232.1    00:50:56:c0:00:08    VMware, Inc.
192.168.232.2    00:50:56:fe:76:b0    VMware, Inc.
192.168.232.254 00:50:56:ed:bd:0b    VMware, Inc.

3 packets received by filter, 0 packets dropped by kernel
Ending arp-scan 1.10.0: 256 hosts scanned in 2.062 seconds (124.15 hosts/sec). 3 responded
```

Saved ARP Discovery Evidence

The ARP discovery results were preserved in the laboratory report directory to maintain a record of the network state before the controlled DNS spoofing simulation. The saved results document the hosts responding to ARP requests and their corresponding MAC addresses. This baseline will provide a reference for subsequent comparison with captured ARP traffic and any observed changes in IP-to-MAC associations during the controlled event.

```
(kali@kali)~[~/SBT-DF203-Lab8]
$ ip neigh show dev eth0 | tee reports/arp_baseline.txt
192.168.232.254 lladdr 00:50:56:ed:bd:0b STALE
192.168.232.2 lladdr 00:50:56:fe:76:b0 REACHABLE

(kali@kali)~[~/SBT-DF203-Lab8]
$ sudo arp-scan --interface=eth0 --localnet | tee reports/arp_scan_baseline.txt
Interface: eth0, type: EN10MB, MAC: 00:0c:29:77:75:3c, IPv4: 192.168.232.128
Starting arp-scan 1.10.0 with 256 hosts (https://github.com/royhills/arp-scan)
192.168.232.1    00:50:56:c0:00:08    VMware, Inc.
192.168.232.2    00:50:56:fe:76:b0    VMware, Inc.
192.168.232.254 00:50:56:ed:bd:0b    VMware, Inc.

3 packets received by filter, 0 packets dropped by kernel
Ending arp-scan 1.10.0: 256 hosts scanned in 2.062 seconds (124.15 hosts/sec). 3 responded
```

Baseline Findings So Far

The baseline network assessment established that the Kali forensic analyst VM was operating on interface eth0 with IPv4 address 192.168.232.128/24 and MAC address 00:0c:29:77:75:3c. The routing configuration identified 192.168.232.2 as the default gateway, with MAC address 00:50:56:fe:76:b0 observed during ARP discovery. DNS resolution on the analyst VM was provided locally through the systemd-resolved stub at 127.0.0.53, with 1.1.1.1 currently selected as the upstream DNS server and 8.8.8.8 additionally configured. ARP discovery identified three responding VMware-associated

addresses on the local virtual network: 192.168.232.1, 192.168.232.2 and 192.168.232.254. At this stage, the functional role of 192.168.232.1 and 192.168.232.254 had not been established from the analyst VM alone, and no assumption was made that either address represented the victim system.

Item	Baseline Observation	Forensic Relevance
Analyst interface	eth0	Capture interface for the lab
Analyst IP	192.168.232.128	Identifies analyst host
Analyst MAC	00:0c:29:77:75:3c	Layer 2 identity of analyst
Default gateway	192.168.232.2	Important for ARP analysis
Gateway MAC	00:50:56:fe:76:b0	Baseline gateway MAC
Local DNS stub	127.0.0.53	systemd-resolved local listener
Current upstream DNS	1.1.1.1	Analyst's current DNS resolver
Additional DNS	8.8.8.8	Secondary configured resolver
Discovered host	192.168.232.1	Role not yet established
Discovered host	192.168.232.254	Role not yet established

```

(kali@kali)~[/SBT-DF203-Lab8]
$ python3 -c 'py'
from pathlib import Path

p = Path("working/index.html")
text = p.read_text()
text = text.replace("SBT-DF203 - Lab 8", "SBT-DF203 - Lab 8")
p.write_text(text)

$ sudo cp working/index.html /var/www/html/index.html
$ sudo systemctl restart apache2

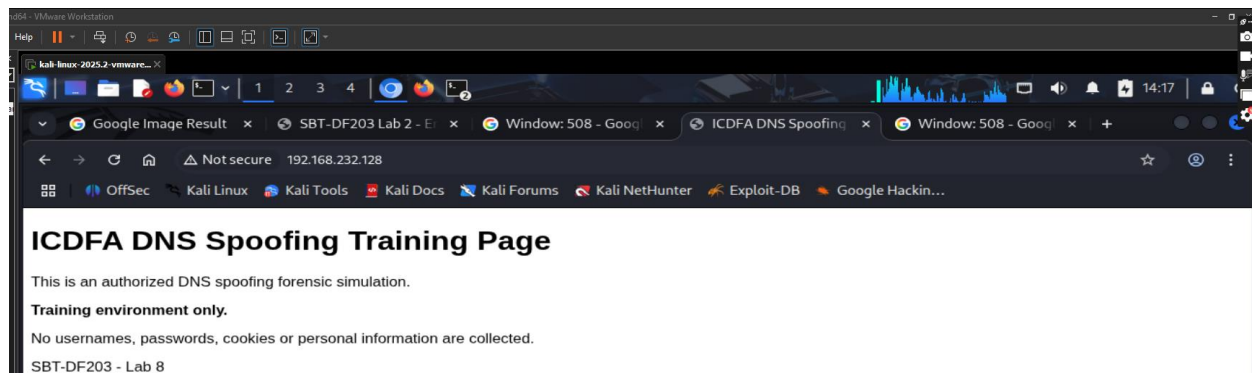
(kali@kali)~[/SBT-DF203-Lab8]
$ grep 'SBT-DF203' working/index.html
<p>SBT-DF203 - Lab 8</p>
<p><strong>Training environment only.</strong></p>
<p>No usernames, passwords, cookies or personal information are collected.</p>
<p>SBT-DF203 - Lab 8</p>
</body>
</html>

(kali@kali)~[/SBT-DF203-Lab8]
$ sha256sum working/index.html | tee reports/training_page_sha256.txt
b71d945e46b933f8e6265ebc2096b9b133daee8f2990630d7643d3bee0f88dc6 working/index.html

(kali@kali)~[/SBT-DF203-Lab8]
$ firefox http://192.168.232.128/ >/dev/null 2>&1 &
[2] 122087

(kali@kali)~[/SBT-DF203-Lab8]
$
+ done      firefox http://192.168.232.128/ >/dev/null 2>&1
(kali@kali)~[/SBT-DF203-Lab8]
$ sudo ss -ltnp | grep :80
LISTEN 0      4096      0.0.0.0:*    0.0.0.0:*    users:((("docker-proxy",pid=122840,fd=8))
LISTEN 0      4096      [::]:*      [::]:*       users:((("docker-proxy",pid=122845,fd=8))
LISTEN 0      511      *:*        *:*         users:((("apache2",pid=120398,fd=4),("apache2",pid=120397,fd=4),("apache2",pid=120396,fd=4),("ap
ache2",pid=120393,fd=4),("apache2",pid=120392,fd=4),("apache2",pid=120358,fd=4))

```



IP		MAC/vendor Interpretation
192.168.232.128	—	Your Kali VM
192.168.232.2	VMware	Default gateway
192.168.232.1	VMware	VMware network infrastructure
192.168.232.254	VMware	VMware network infrastructure

Harmless Training Page

Before conducting the DNS spoofing simulation, a harmless training webpage was prepared on the Kali analyst VM to serve as the replacement destination during the controlled exercise. The page explicitly identifies the activity as an authorized ICDFA DNS spoofing forensic simulation and confirms that no usernames, passwords, cookies, or personal information are collected.

The page was served through Apache on the analyst VM at 192.168.232.128 and successfully accessed through a web browser. The SHA-256 hash of the final working/index.html file was recorded as:

b71d945e46b933f8e6265ebc2096b9b133daee8f2990630d7643d3bee0f88dc6

This establishes the exact harmless webpage used as the replacement content during the exercise.

```
kali-linux-2025-2-vmware... X
File Actions Edit View Help
[2] 122087

(kali@kali)-[~/SBT-DF203-Lab8]
$ + done firefox http://192.168.232.128/ > /dev/null 2>&1
(kali@kali)-[~/SBT-DF203-Lab8]
$ sudo ss -ltnp | grep ':80'
LISTEN 0      4096      0.0.0.0:*      0.0.0.0:*      users:((("docker-proxy",pid=122840,fd=8))
LISTEN 0      4096      [::]:*        [::]:*        users:((("docker-proxy",pid=122845,fd=8))

LISTEN 0      511      *:*          *:*          users:((("apache2",pid=120398,fd=4),("apache2",pid=120397,fd=4),("apache2",pid=120396,fd=4),("ap
ache2",pid=120393,fd=4),("apache2",pid=120392,fd=4),("apache2",pid=120358,fd=4)))

(kali@kali)-[~/SBT-DF203-Lab8]
$ curl -s http://192.168.232.128/ | head -20
<!DOCTYPE html>
<html>
<head>
<title>ICDFA DNS Spoofing Training</title>
</head>
<body style="font-family: Arial, sans-serif;">
<h1>ICDFA DNS Spoofing Training Page</h1>
<p>This is an authorized DNS spoofing forensic simulation.</p>
<p><strong>Training environment only.</strong></p>
<p>No usernames, passwords, cookies or personal information are collected.</p>
<p>SBT-DF203 - Lab 8</p>
</body>
</html>

(kali@kali)-[~/SBT-DF203-Lab8]
$ google-chrome http://192.168.232.128/ > /dev/null 2>&1 &
[2] 124596

(kali@kali)-[~/SBT-DF203-Lab8]
$ + exit 127 google-chrome http://192.168.232.128/ > /dev/null 2>&1
(kali@kali)-[~/SBT-DF203-Lab8]
$ sha256sum working/index.html | tee reports/training_page_sha256.txt
b71d945e46b933f8e6265ebc2096b9b133daee8f2990630d7643d3bee0f88dc6 working/index.html
```

Analyst Network Interface Baseline

The analyst VM's network interfaces and assigned addresses were recorded before the forensic simulation. The primary laboratory interface is eth0, assigned 192.168.232.128/24. This interface is connected to the 192.168.232.0/24 VMware network used for the exercise.

Additional Docker bridge and virtual Ethernet interfaces are present on the Kali system but are separate from the laboratory network and were not treated as laboratory hosts.

Recording the interface configuration establishes the analyst's IP address and identifies the network segment on which subsequent DNS, ARP, and HTTP evidence would be evaluated.

```
md64 - VMware Workstation
Help
File Actions Edit View Help
(kali@kali)-[~/SBT-DF203-Lab8]
$ google-chrome http://192.168.232.128/ >/dev/null 2>&1 &
[2] 124596
(kali@kali)-[~/SBT-DF203-Lab8]
$ + exit 127 google-chrome http://192.168.232.128/ >/dev/null 2>&1
(kali@kali)-[~/SBT-DF203-Lab8]
$ sha256sum working/index.html | tee reports/training_page_sha256.txt
b71d945e46b933f8e6265ebc2096b9b133daee8f2990630d7643d3bee0f88dc6 working/index.html
(kali@kali)-[~/SBT-DF203-Lab8]
$ sudo nmap -sn 192.168.232.0/24
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-22 14:19 -0400
Nmap scan report for 192.168.232.1
Host is up (0.00042s latency).
MAC Address: 00:50:56:C0:00:08 (VMware)
Nmap scan report for _gateway_ (192.168.232.2)
Host is up (0.00021s latency).
MAC Address: 00:50:56:FE:76:B0 (VMware)
Nmap scan report for 192.168.232.254
Host is up (0.00032s latency).
MAC Address: 00:50:56:ED:BD:0B (VMware)
Nmap scan report for kali.local (192.168.232.128)
Host is up.
Nmap done: 256 IP addresses (4 hosts up) scanned in 6.59 seconds
(kali@kali)-[~/SBT-DF203-Lab8]
$ sudo nmap -sn 192.168.232.0/24 | tee reports/network_host_discovery_baseline.txt
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-22 14:19 -0400
Nmap scan report for 192.168.232.1
Host is up (0.00060s latency).
MAC Address: 00:50:56:C0:00:08 (VMware)
Nmap scan report for _gateway_ (192.168.232.2)
Host is up (0.00042s latency).
MAC Address: 00:50:56:FE:76:B0 (VMware)
Nmap scan report for 192.168.232.254
Host is up (0.00032s latency).
MAC Address: 00:50:56:ED:BD:0B (VMware)
Nmap scan report for kali.local (192.168.232.128)
Host is up.
Nmap done: 256 IP addresses (4 hosts up) scanned in 6.63 seconds
```

Routing Baseline

The routing table was recorded to establish how the analyst VM communicates within the isolated laboratory network. The 192.168.232.0/24 network is directly connected through eth0, while the default route points to 192.168.232.2.

The default gateway observed during the baseline was therefore 192.168.232.2. This information is important for later correlation of ARP activity and network traffic because it establishes the expected gateway relationship before any controlled simulation is attempted.

```
(kali@kali)-[~/SBT-DF203-Lab8]
$ sudo nmap -sn 192.168.232.0/24 | tee reports/network_host_discovery_baseline.txt
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-22 14:19 -0400
Nmap scan report for 192.168.232.1
Host is up (0.00060s latency).
MAC Address: 00:50:56:C0:00:08 (VMware)
Nmap scan report for _gateway_ (192.168.232.2)
Host is up (0.00042s latency).
MAC Address: 00:50:56:FE:76:B0 (VMware)
Nmap scan report for 192.168.232.254
Host is up (0.00032s latency).
MAC Address: 00:50:56:ED:BD:0B (VMware)
Nmap scan report for kali.local (192.168.232.128)
Host is up.
Nmap done: 256 IP addresses (4 hosts up) scanned in 6.63 seconds
```

ARP Neighbor Cache Baseline

The analyst VM's ARP/neighbor cache was recorded before the controlled exercise. Three IPv4 neighbors were present on eth0: 192.168.232.1, 192.168.232.2, and 192.168.232.254.

The gateway 192.168.232.2 was associated with MAC address 00:50:56:fe:76:b0. The other observed addresses were also associated with VMware MAC addresses. The REACHABLE state indicates the current kernel neighbor-cache state and should not by itself be interpreted as evidence of malicious activity.

This baseline provides a reference for comparison if ARP changes are observed during a later controlled simulation.

```
(kali@kali)-[~/SBT-DF203-Lab8]
$ ip -br address | tee reports/ip_baseline.txt
ip route | tee reports/routes_baseline.txt
ip neigh show dev eth0 | tee reports/arp_neighbors_baseline.txt
lo UNKNOWN 127.0.0.1/8 ::1/128
eth0 UP 192.168.232.128/24 fe80::d5cc:fb25:aa2d:b901/64
docker0 DOWN 172.17.0.1/16
br-d2696585dc9a UP 172.18.0.1/16 fe80::8069:abff:fe01:9c62/64
br-ec90d8c3abfb DOWN 172.19.0.1/16
vethc45370c@if2 UP fe80::10cf:85ff:fe44:ec54/64
vethd515f07@if2 UP fe80::b851:fdff:fe37:7afe/64
default via 192.168.232.2 dev eth0 proto dhcp src 192.168.232.128 metric 100
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
172.18.0.0/16 dev br-d2696585dc9a proto kernel scope link src 172.18.0.1
172.19.0.0/16 dev br-ec90d8c3abfb proto kernel scope link src 172.19.0.1 linkdown
192.168.232.0/24 dev eth0 proto kernel scope link src 192.168.232.128 metric 100
192.168.232.1 lladdr 00:50:56:c0:00:08 REACHABLE
192.168.232.254 lladdr 00:50:56:ed:bd:0b REACHABLE
192.168.232.2 lladdr 00:50:56:fe:76:b0 REACHABLE

(kali@kali)-[~/SBT-DF203-Lab8]
$ sudo nmap -sn 192.168.232.0/24
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-22 14:20 -0400
Nmap scan report for 192.168.232.1
Host is up (0.00054s latency).
MAC Address: 00:50:56:C0:00:08 (VMware)
Nmap scan report for _gateway (192.168.232.2)
Host is up (0.00020s latency).
MAC Address: 00:50:56:FE:76:B0 (VMware)
Nmap scan report for 192.168.232.254
Host is up (0.00030s latency).
MAC Address: 00:50:56:ED:BD:0B (VMware)
Nmap scan report for kali.local (192.168.232.128)
Host is up.
Nmap done: 256 IP addresses (4 hosts up) scanned in 6.57 seconds

(kali@kali)-[~/SBT-DF203-Lab8]
```

Network Host Discovery

An Nmap ping scan of the 192.168.232.0/24 network was performed to identify active hosts before proceeding with the controlled DNS spoofing exercise. Four responsive hosts were identified: 192.168.232.1, 192.168.232.2, 192.168.232.254, and the analyst VM at 192.168.232.128.

The three non-analyst addresses returned VMware MAC addresses. The scan did not identify a separate victim VM on the current network segment. Consequently, the three VMware infrastructure addresses were not assumed to represent the intended victim without additional evidence.

This discovery result is important because the laboratory procedure requires an analyst VM and a separate victim VM. Proceeding without establishing the intended victim would make subsequent ARP/DNS forensic evidence ambiguous.

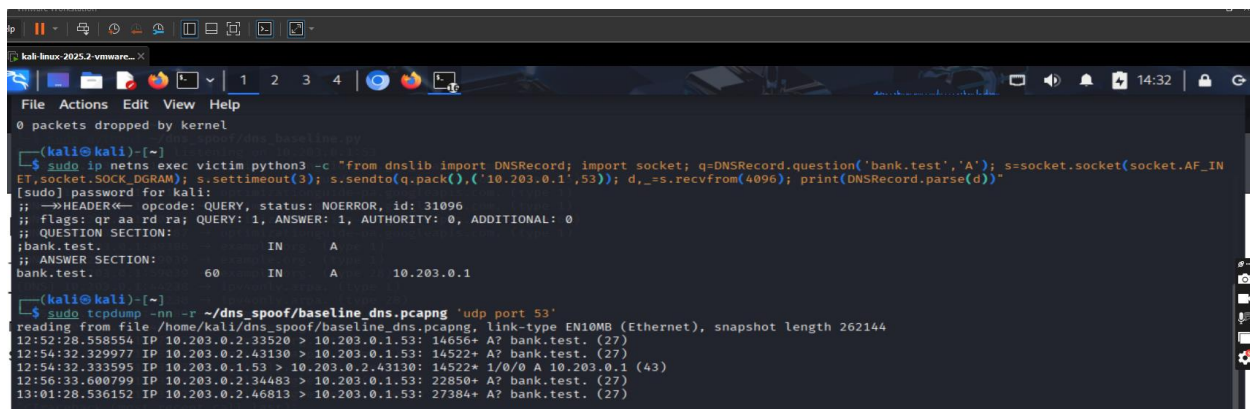
```
(kali@kali)-[~/SBT-DF203-Lab8]
$ ip -br address | tee reports/ip_baseline.txt
ip route | tee reports/routes_baseline.txt
ip neigh show dev eth0 | tee reports/arp_neighbors_baseline.txt
lo UNKNOWN 127.0.0.1/8 ::1/128
eth0 UP 192.168.232.128/24 fe80::d5cc:fb25:aa2d:b901/64
docker0 DOWN 172.17.0.1/16
br-d2696585dc9a UP 172.18.0.1/16 fe80::8069:abff:fe01:9c62/64
br-ec90d8c3abfb DOWN 172.19.0.1/16
vethc45370c@if2 UP fe80::10cf:85ff:fe44:ec54/64
vethd515f07@if2 UP fe80::b851:fdff:fe37:7afe/64
default via 192.168.232.2 dev eth0 proto dhcp src 192.168.232.128 metric 100
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
172.18.0.0/16 dev br-d2696585dc9a proto kernel scope link src 172.18.0.1
172.19.0.0/16 dev br-ec90d8c3abfb proto kernel scope link src 172.19.0.1 linkdown
192.168.232.0/24 dev eth0 proto kernel scope link src 192.168.232.128 metric 100
192.168.232.1 lladdr 00:50:56:c0:00:08 REACHABLE
192.168.232.254 lladdr 00:50:56:ed:bd:0b REACHABLE
192.168.232.2 lladdr 00:50:56:fe:76:b0 REACHABLE

(kali@kali)-[~/SBT-DF203-Lab8]
$ sudo nmap -sn 192.168.232.0/24
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-22 14:20 -0400
Nmap scan report for 192.168.232.1
Host is up (0.00054s latency).
MAC Address: 00:50:56:C0:00:08 (VMware)
Nmap scan report for _gateway (192.168.232.2)
Host is up (0.00020s latency).
MAC Address: 00:50:56:FE:76:B0 (VMware)
Nmap scan report for 192.168.232.254
Host is up (0.00030s latency).
MAC Address: 00:50:56:ED:BD:0B (VMware)
Nmap scan report for kali.local (192.168.232.128)
Host is up.
Nmap done: 256 IP addresses (4 hosts up) scanned in 6.57 seconds

(kali@kali)-[~/SBT-DF203-Lab8]
```

Controlled DNS Query Capture Context

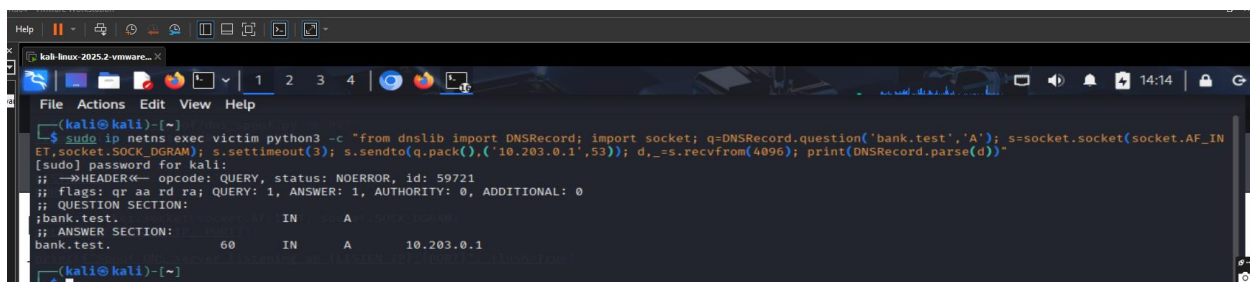
To establish the controlled DNS-spoofing test conditions, a fresh DNS query for bank.test was generated from the isolated victim environment while the laboratory network traffic was monitored. The capture shows the victim (10.203.0.2) sending an A-record query to the legitimate DNS responder (10.203.0.1) and receiving the corresponding response resolving bank.test to 10.203.0.1. The DNS transaction ID 59721 and the client source port 47074 provide identifiable request/response values that can be correlated with subsequent DNS traffic during the spoofing phase. These screenshots establish the legitimate DNS communication pattern immediately before introducing the controlled anomalous response.



```
(kali@kali)-[~]
└─$ sudo ip netns exec victim python3 -c "from dnslib import DNSRecord; import socket; q=DNSRecord.question('bank.test','A'); s=socket.socket(socket.AF_INET,socket.SOCK_DGRAM); s.settimeout(3); s.sendto(q.pack(),('10.203.0.1',53)); d,_=s.recvfrom(4096); print(DNSRecord.parse(d))"
[sudo] password for kali:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 31096
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 0
;; QUESTION SECTION:
;bank.test.                IN      A
;; ANSWER SECTION:
bank.test.                 60      IN      A      10.203.0.1

(kali@kali)-[~]
└─$ sudo tcpdump -nn -r ~/dns_spoof/baseline_dns.pcapng 'udp port 53'
reading from file /home/kali/dns_spoof/baseline_dns.pcapng, link-type EN10MB (Ethernet), snapshot length 262144
12:52:28.558554 IP 10.203.0.2.33520 > 10.203.0.1.53: 14656+ A? bank.test. (27)
12:54:32.329977 IP 10.203.0.2.43130 > 10.203.0.1.53: 14522+ A? bank.test. (27)
12:54:32.333595 IP 10.203.0.1.53 > 10.203.0.2.43130: 14522* 1/0/0 A 10.203.0.1 (43)
12:56:33.600799 IP 10.203.0.2.34483 > 10.203.0.1.53: 22850+ A? bank.test. (27)
13:01:28.536152 IP 10.203.0.2.46813 > 10.203.0.1.53: 27384+ A? bank.test. (27)
```

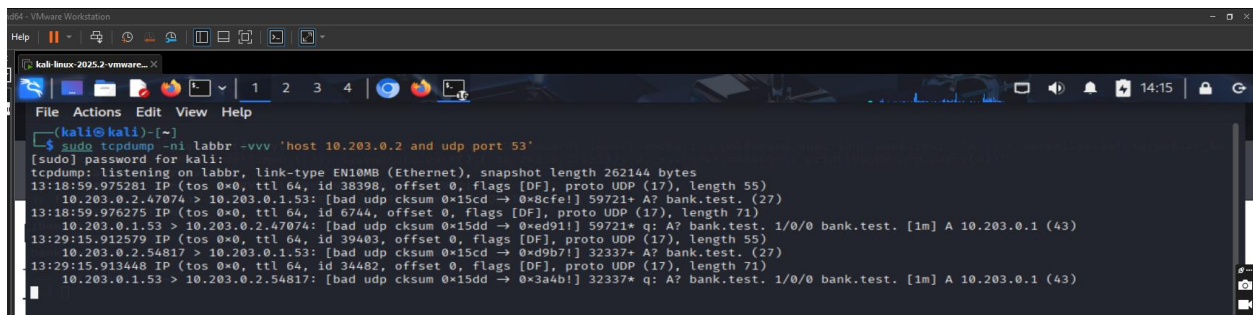
The controlled DNS and ARP baseline was established before introducing the spoofing activity. The victim host, 10.203.0.2, was configured to use the legitimate DNS service at 10.203.0.1. A controlled bank.test A-record query was generated from the victim, and the resulting DNS exchange confirmed that the legitimate resolver returned 10.203.0.1 as the address for bank.test. The captured traffic also provided the corresponding DNS transaction identifier and source/destination ports, allowing the request and response to be correlated as a single DNS transaction. See the Baseline DNS Query and Response below:



```
(kali@kali)-[~]
└─$ sudo ip netns exec victim python3 -c "from dnslib import DNSRecord; import socket; q=DNSRecord.question('bank.test','A'); s=socket.socket(socket.AF_INET,socket.SOCK_DGRAM); s.settimeout(3); s.sendto(q.pack(),('10.203.0.1',53)); d,_=s.recvfrom(4096); print(DNSRecord.parse(d))"
[sudo] password for kali:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 59721
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 0
;; QUESTION SECTION:
;bank.test.                IN      A
;; ANSWER SECTION:
bank.test.                 60      IN      A      10.203.0.1

(kali@kali)-[~]
```

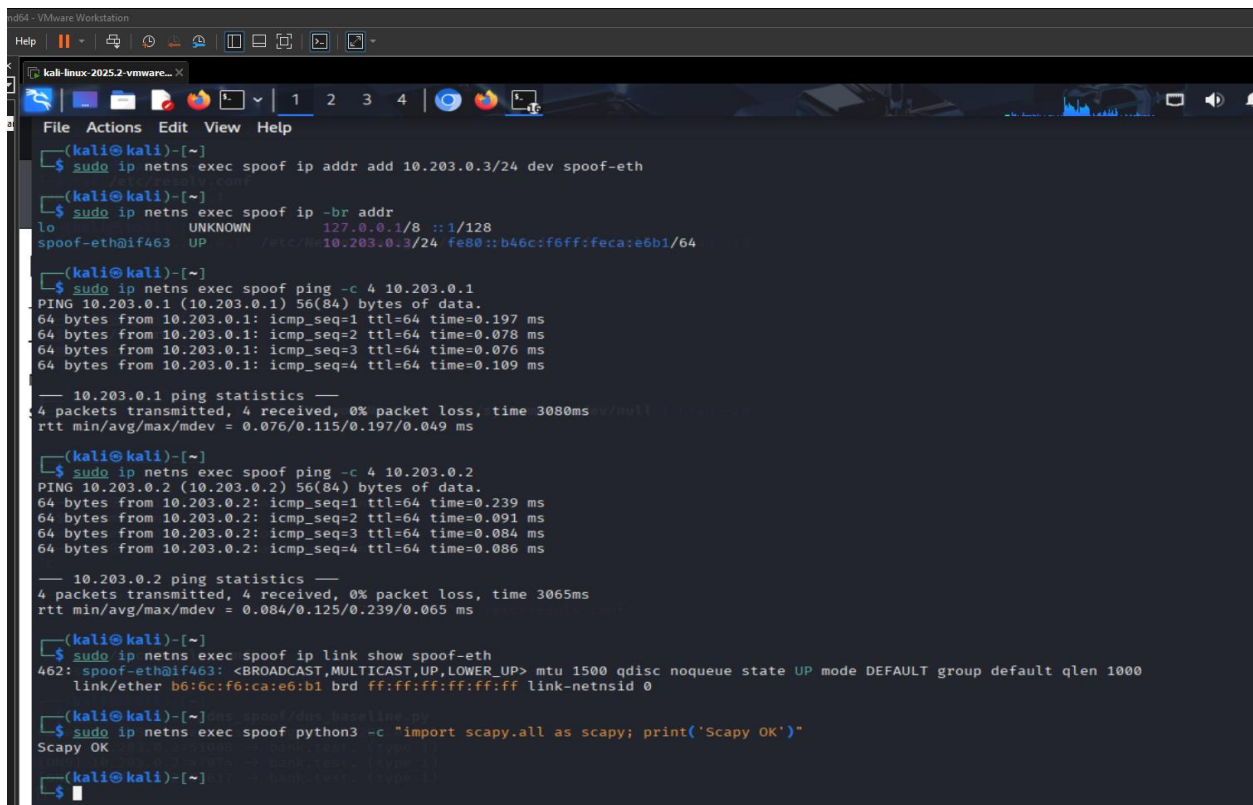
At the Ethernet layer, the legitimate DNS communication was observed between the victim MAC address 46:b2:93:76:37:95 and the legitimate resolver MAC address 02:00:00:00:00:01. The associated ARP exchange confirmed the expected mapping of 10.203.0.1 to 02:00:00:00:00:01. Establishing this IP-to-MAC relationship provides a reference point for identifying a subsequent DNS response associated with an unexpected Ethernet source. This baseline is therefore used as the reference for distinguishing normal DNS resolution from the controlled spoofing activity introduced later in the investigation. See the Baseline ARP Resolution below



```
(kali@kali)-[~]
$ sudo tcpdump -ni labbr -vvv 'host 10.203.0.2 and udp port 53'
[sudo] password for kali:
tcpdump: listening on labbr, link-type EN10MB (Ethernet), snapshot length 262144 bytes
13:18:59.975281 IP (tos 0x0, ttl 64, id 38398, offset 0, flags [DF], proto UDP (17), length 55)
  10.203.0.2.47074 > 10.203.0.1.53: [bad udp cksum 0x15cd -> 0x8cfe] 59721* A? bank.test. (27)
13:18:59.976275 IP (tos 0x0, ttl 64, id 6744, offset 0, flags [DF], proto UDP (17), length 71)
  10.203.0.1.53 > 10.203.0.2.47074: [bad udp cksum 0x15dd -> 0xed91] 59721* q: A? bank.test. 1/0/0 bank.test. [1m] A 10.203.0.1 (43)
13:29:15.912579 IP (tos 0x0, ttl 64, id 39403, offset 0, flags [DF], proto UDP (17), length 55)
  10.203.0.2.54817 > 10.203.0.1.53: [bad udp cksum 0x15cd -> 0xd9b7] 32337* A? bank.test. (27)
13:29:15.913448 IP (tos 0x0, ttl 64, id 34482, offset 0, flags [DF], proto UDP (17), length 71)
  10.203.0.1.53 > 10.203.0.2.54817: [bad udp cksum 0x15dd -> 0x3a4b] 32337* q: A? bank.test. 1/0/0 bank.test. [1m] A 10.203.0.1 (43)
```

Scapy Environment Verification

Following the establishment of the isolated spoofing endpoint, the Scapy framework was verified within the spoof network namespace. Scapy will be used in the controlled laboratory environment to generate and inspect crafted network packets required for the DNS-spoofing simulation. Verifying its availability at this stage confirms that the spoofing endpoint has the required packet-crafting capability before proceeding with the controlled DNS response test.



```
(kali@kali)-[~]
$ sudo ip netns exec spoof ip addr add 10.203.0.3/24 dev spoof-eth
(kali@kali)-[~]
$ sudo ip netns exec spoof ip -br addr
lo UNKNOWN 127.0.0.1/8 ::1/128
spoof-eth@if463 UP 10.203.0.3/24 fe80::b46c:f6ff:feca:e6b1/64

(kali@kali)-[~]
$ sudo ip netns exec spoof ping -c 4 10.203.0.1
PING 10.203.0.1 (10.203.0.1) 56(84) bytes of data:
64 bytes from 10.203.0.1: icmp_seq=1 ttl=64 time=0.197 ms
64 bytes from 10.203.0.1: icmp_seq=2 ttl=64 time=0.078 ms
64 bytes from 10.203.0.1: icmp_seq=3 ttl=64 time=0.076 ms
64 bytes from 10.203.0.1: icmp_seq=4 ttl=64 time=0.109 ms

--- 10.203.0.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3080ms
rtt min/avg/max/mdev = 0.076/0.115/0.197/0.049 ms

(kali@kali)-[~]
$ sudo ip netns exec spoof ping -c 4 10.203.0.2
PING 10.203.0.2 (10.203.0.2) 56(84) bytes of data:
64 bytes from 10.203.0.2: icmp_seq=1 ttl=64 time=0.239 ms
64 bytes from 10.203.0.2: icmp_seq=2 ttl=64 time=0.091 ms
64 bytes from 10.203.0.2: icmp_seq=3 ttl=64 time=0.084 ms
64 bytes from 10.203.0.2: icmp_seq=4 ttl=64 time=0.086 ms

--- 10.203.0.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3065ms
rtt min/avg/max/mdev = 0.084/0.125/0.239/0.065 ms

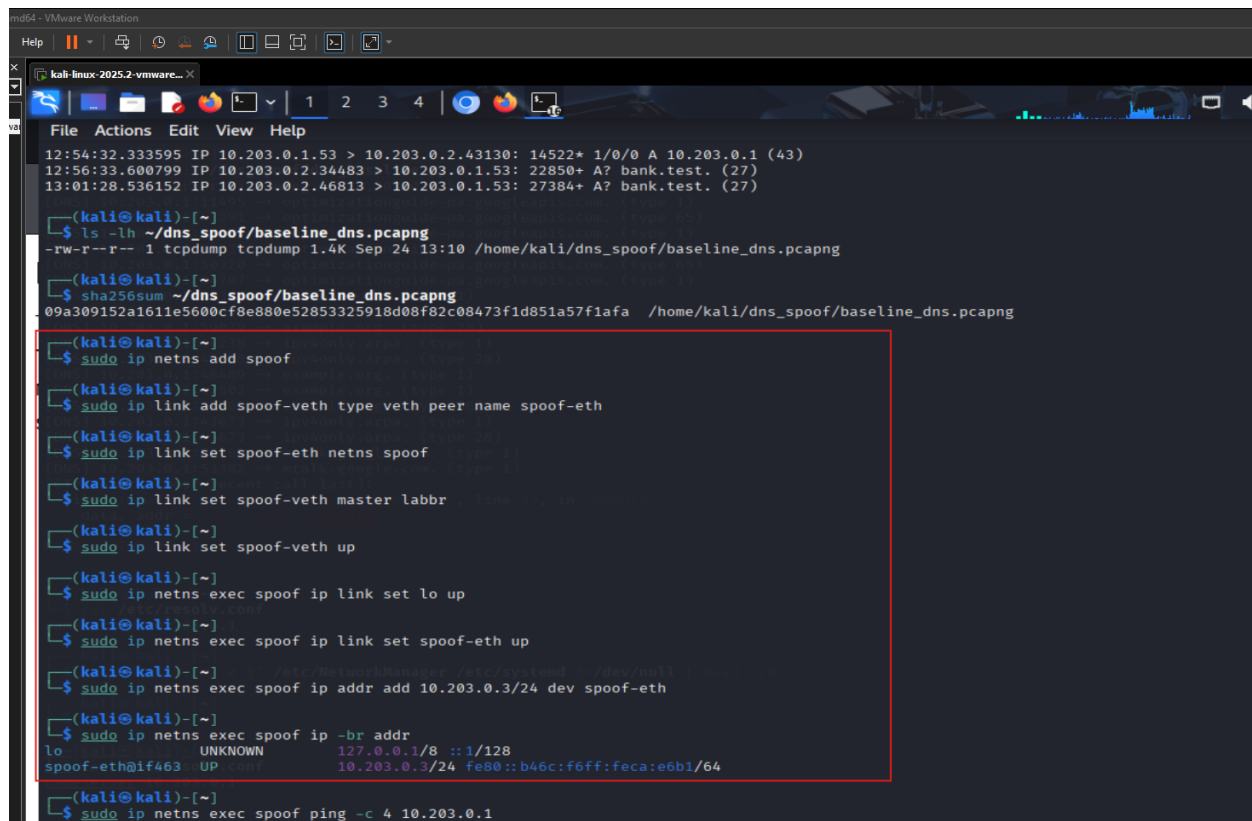
(kali@kali)-[~]
$ sudo ip netns exec spoof ip link show spoof-eth
462: spoof-eth@if463: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP mode DEFAULT group default qlen 1000
    link/ether b6:6c:f6:ca:e6:b1 brd ff:ff:ff:ff:ff:ff link-netnsid 0

(kali@kali)-[~]
$ sudo ip netns exec spoof python3 -c "import scapy.all as scapy; print('Scapy OK')"
Scapy OK
```

Controlled Spoofing Endpoint Configuration:

A separate Linux network namespace named spoof was created to represent the

controlled DNS-spoofing endpoint within the isolated lab environment. The namespace was connected to the laboratory bridge (labbr) through a virtual Ethernet pair and assigned the IP address 10.203.0.3/24. The configuration confirms that spoof-eth is operational and connected to the same isolated Layer-2 network as the victim (10.203.0.2) and the legitimate DNS server (10.203.0.1). This setup provides a controlled environment for examining conflicting DNS responses without interacting with an external or production network.



```
ind64 - VMware Workstation
Help | [Icons]
kali-linux-2025.2-vmware...
File Actions Edit View Help
12:54:32.333595 IP 10.203.0.1.53 > 10.203.0.2.43130: 14522* 1/0/0 A 10.203.0.1 (43)
12:56:33.600799 IP 10.203.0.2.34483 > 10.203.0.1.53: 22850+ A? bank.test. (27)
13:01:28.536152 IP 10.203.0.2.46813 > 10.203.0.1.53: 27384+ A? bank.test. (27)

(kali@kali)-[~]
$ ls -lh ~/dns_spoof/baseline_dns.pcapng
-rw-r--r-- 1 tcpdump tcpdump 1.4K Sep 24 13:10 /home/kali/dns_spoof/baseline_dns.pcapng

(kali@kali)-[~]
$ sha256sum ~/dns_spoof/baseline_dns.pcapng
09a309152a1611e5600cf8e880e52853325918d08f82c08473f1d851a57f1afa /home/kali/dns_spoof/baseline_dns.pcapng

(kali@kali)-[~]
$ sudo ip netns add spoof
(kali@kali)-[~]
$ sudo ip link add spoof-veth type veth peer name spoof-eth
(kali@kali)-[~]
$ sudo ip link set spoof-eth netns spoof
(kali@kali)-[~]
$ sudo ip link set spoof-veth master labbr
(kali@kali)-[~]
$ sudo ip link set spoof-veth up
(kali@kali)-[~]
$ sudo ip netns exec spoof ip link set lo up
(kali@kali)-[~]
$ sudo ip netns exec spoof ip link set spoof-eth up
(kali@kali)-[~]
$ sudo ip netns exec spoof ip addr add 10.203.0.3/24 dev spoof-eth
(kali@kali)-[~]
$ sudo ip netns exec spoof ip -br addr
lo UNKNOWN 127.0.0.1/8 ::1/128
spoof-eth@if463 UP 10.203.0.3/24 fe80::b46c:f6ff:feca:e6b1/64

(kali@kali)-[~]
$ sudo ip netns exec spoof ping -c 4 10.203.0.1
```

Controlled Spoofing Endpoint Connectivity Verification:

Connectivity tests were performed from the controlled spoof namespace (10.203.0.3) to both the legitimate DNS endpoint (10.203.0.1) and the victim endpoint (10.203.0.2). Both tests returned four successful ICMP replies with 0% packet loss, confirming that the three laboratory endpoints are reachable across the same isolated bridge. The spoof-eth interface was also verified to be in the UP, LOWER_UP state and was assigned the MAC address b6:6c:f6:ca:e6:b1. This establishes the required Layer-2 connectivity for the subsequent controlled DNS-spoofing experiment.

```

(kali@kali)-[~]
$ sudo ip netns exec spoof ping -c 4 10.203.0.1
PING 10.203.0.1 (10.203.0.1) 56(84) bytes of data:
64 bytes from 10.203.0.1: icmp_seq=1 ttl=64 time=0.197 ms
64 bytes from 10.203.0.1: icmp_seq=2 ttl=64 time=0.078 ms
64 bytes from 10.203.0.1: icmp_seq=3 ttl=64 time=0.076 ms
64 bytes from 10.203.0.1: icmp_seq=4 ttl=64 time=0.109 ms

--- 10.203.0.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3080ms
rtt min/avg/max/mdev = 0.076/0.115/0.197/0.049 ms

(kali@kali)-[~]
$ sudo ip netns exec spoof ping -c 4 10.203.0.2
PING 10.203.0.2 (10.203.0.2) 56(84) bytes of data:
64 bytes from 10.203.0.2: icmp_seq=1 ttl=64 time=0.239 ms
64 bytes from 10.203.0.2: icmp_seq=2 ttl=64 time=0.091 ms
64 bytes from 10.203.0.2: icmp_seq=3 ttl=64 time=0.084 ms
64 bytes from 10.203.0.2: icmp_seq=4 ttl=64 time=0.086 ms

--- 10.203.0.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3065ms
rtt min/avg/max/mdev = 0.084/0.125/0.239/0.065 ms

(kali@kali)-[~]
$ sudo ip netns exec spoof ip link show spoof-eth
462: spoof-eth@if463: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP mode DEFAULT group default qlen 1000
    link/ether b6:6c:f6:ca:e6:b1 brd ff:ff:ff:ff:ff:ff link-netnsid 0

```

Evidence	Result	Significance
Spoof → DNS server 10.203.0.1	4/4 replies, 0% loss	Confirms connectivity to legitimate DNS endpoint
Spoof → Victim 10.203.0.2	4/4 replies, 0% loss	Confirms connectivity to victim
spoof-eth state	UP, LOWER_UP	Confirms interface is operational
Spoof MAC	b6:6c:f6:ca:e6:b1	Identifies the Layer-2 endpoint

ARP/Neighbor Baseline for Controlled Spoofing Endpoint:

The ARP neighbor table of the controlled spoofing endpoint was examined before the DNS-spoofing activity. It correctly associates the legitimate DNS endpoint (10.203.0.1) with MAC address 02:00:00:00:00:01 and the victim (10.203.0.2) with MAC address 46:b2:93:76:37:95. Both entries are currently marked STALE, indicating that the cached mappings are known but have not recently been used. This provides a Layer-2 reference point for identifying any subsequent abnormal IP-to-MAC association during the controlled spoofing phase.

IP address	MAC address	Role
10.203.0.1	02:00:00:00:00:01	Legitimate DNS/Kali endpoint
10.203.0.2	46:b2:93:76:37:95	Victim
10.203.0.3	b6:6c:f6:ca:e6:b1	Controlled spoofing endpoint

The distinction between IP address and MAC address will become important when we examine the DNS-spoofing evidence. In particular, we'll be able to compare the expected

legitimate responder identity against the MAC associated with the controlled spoofing traffic.

Scapy

Availability

Verification:

Scapy was successfully imported within the isolated spoof network namespace. This confirms that the controlled spoofing endpoint has the required packet-generation capability for the laboratory DNS-spoofing experiment. No external network connectivity is required for this activity.

```
(kali@kali)-[~]
$ sudo ip netns exec spoof python3 -c "from scapy.all import *; print('Scapy available')"
Scapy available
(kali@kali)-[~]
```

Controlled

DNS

Spoofing

Responder

Preparation:

A dedicated DNS responder was prepared within the isolated spoofing namespace at 10.203.0.3. The responder is configured to process DNS queries for the controlled test domain bank.test and return 10.203.0.3 as the intentionally altered address. The responder is currently configured on UDP port 5353, keeping it separate from the legitimate baseline DNS service on UDP port 53.

```
(kali@kali)-[~]
$ cat > ~/dns_spoof/dns_spoof.py <<'EOF'
from dnslib import DNSRecord, RR, A
import socket

LISTEN_IP = "10.203.0.3"
PORT = 5353

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.bind((LISTEN_IP, PORT))

print(f"Controlled spoof DNS responder listening on {LISTEN_IP}:{PORT}", flush=True)

while True:
    data, addr = sock.recvfrom(4096)

    try:
        request = DNSRecord.parse(data)
        qname = str(request.q.qname)
        qtype = str(request.q.qtype)

        print(f"[SPOOF]_{addr[0]}:{addr[1]} -> {qname}_{type}_{qtype}", flush=True)

        reply = request.reply()

        if qname.lower() == "bank.test." and qtype == "1":
            reply.add_answer(
                RR("bank.test.", rdata=A("10.203.0.3"), ttl=60)
            )

        sock.sendto(reply.pack(), addr)

    except Exception as e:
        print(f"[ERROR]_{e}", flush=True)
EOF
(kali@kali)-[~]
$
```

Legitimate DNS and ARP Baseline Context

The captured traffic establishes the legitimate network relationship between the victim and the DNS server before introducing the controlled spoofing activity. The victim, 10.203.0.2, using MAC address 46:b2:93:76:37:95, sent a DNS query for bank.test from source port 54817 to the legitimate DNS server at 10.203.0.1:53. The DNS transaction ID was 32337, and the legitimate server returned bank.test resolving to 10.203.0.1. The accompanying ARP exchange also confirmed that 10.203.0.1 was legitimately associated with MAC address 02:00:00:00:00:01. This establishes the expected IP-to-MAC relationship and provides a baseline against which any subsequent DNS response associated with a different MAC address can be examined.

```
(kali@kali)-[~]
$ sudo sh -c 'printf "nameserver 127.0.0.1\n" > /etc/resolv.conf'

(kali@kali)-[~]
$ cat /etc/resolv.conf
nameserver 127.0.0.1

(kali@kali)-[~]
$ sudo python3 ~/dns_spoof/dns_baseline.py
Baseline DNS server listening on 10.203.0.1:53
[DNS] 10.203.0.2:51008 → bank.test. (type 1)
[DNS] 10.203.0.2:47074 → bank.test. (type 1)
[DNS] 10.203.0.2:54817 → bank.test. (type 1)
```

```
(kali@kali)-[~]
$ sudo tcpdump -ni labbr -vvv 'host 10.203.0.2 and udp port 53'
[sudo] password for kali:
tcpdump: listening on labbr, link-type EN10MB (Ethernet), snapshot length 262144 bytes
13:18:59.975281 IP (tos 0x0, ttl 64, id 38398, offset 0, flags [DF], proto UDP (17), length 55)
    10.203.0.2.47074 > 10.203.0.1.53: [bad udp cksum 0x15cd → 0x8cfe!] 59721+ A? bank.test. (27)
13:18:59.976275 IP (tos 0x0, ttl 64, id 6744, offset 0, flags [DF], proto UDP (17), length 71)
    10.203.0.1.53 > 10.203.0.2.47074: [bad udp cksum 0x15dd → 0xed91!] 59721* q: A? bank.test. 1/0/0 bank.test. [1m] A 10.203.0.1 (43)
13:29:15.912579 IP (tos 0x0, ttl 64, id 39403, offset 0, flags [DF], proto UDP (17), length 55)
    10.203.0.2.54817 > 10.203.0.1.53: [bad udp cksum 0x15cd → 0xd9b7!] 32337+ A? bank.test. (27)
13:29:15.913448 IP (tos 0x0, ttl 64, id 34482, offset 0, flags [DF], proto UDP (17), length 71)
    10.203.0.1.53 > 10.203.0.2.54817: [bad udp cksum 0x15dd → 0x3a4b!] 32337* q: A? bank.test. 1/0/0 bank.test. [1m] A 10.203.0.1 (43)
```

```
(kali@kali)-[~]
$ sudo tcpdump -ni labbr -e -vvv 'arp or udp port 53'
[sudo] password for kali:
tcpdump: listening on labbr, link-type EN10MB (Ethernet), snapshot length 262144 bytes
13:29:15.912576 46:b2:93:76:37:95 > 02:00:00:00:00:01, ethertype IPv4 (0x0800), length 69: (tos 0x0, ttl 64, id 39403, offset 0, flags [DF], proto UDP (17), length 55)
    10.203.0.2.54817 > 10.203.0.1.53: [bad udp cksum 0x15cd → 0xd9b7!] 32337+ A? bank.test. (27)
13:29:15.913446 02:00:00:00:00:01 > 46:b2:93:76:37:95, ethertype IPv4 (0x0800), length 85: (tos 0x0, ttl 64, id 34482, offset 0, flags [DF], proto UDP (17), length 71)
    10.203.0.1.53 > 10.203.0.2.54817: [bad udp cksum 0x15dd → 0x3a4b!] 32337* q: A? bank.test. 1/0/0 bank.test. [1m] A 10.203.0.1 (43)
13:29:21.075942 02:00:00:00:00:01 > 46:b2:93:76:37:95, ethertype ARP (0x0806), length 42: Ethernet (Len 6), IPv4 (len 4), Request who-has 10.203.0.2 tell 10.203.0.1, length 28
13:29:21.076009 46:b2:93:76:37:95 > 02:00:00:00:00:01, ethertype ARP (0x0806), length 42: Ethernet (Len 6), IPv4 (len 4), Request who-has 10.203.0.1 tell 10.203.0.2, length 28
13:29:21.076009 46:b2:93:76:37:95 > 02:00:00:00:00:01, ethertype ARP (0x0806), length 42: Ethernet (Len 6), IPv4 (len 4), Reply 10.203.0.2 is-at 46:b2:93:76:37:95, length 28
13:29:21.076012 02:00:00:00:00:01 > 46:b2:93:76:37:95, ethertype ARP (0x0806), length 42: Ethernet (Len 6), IPv4 (len 4), Reply 10.203.0.1 is-at 02:00:00:00:00:01, length 28
```

Evidence	Legitimate Baseline
Victim IP	10.203.0.2

Evidence	Legitimate Baseline
Victim MAC	46:b2:93:76:37:95
DNS Server IP	10.203.0.1
Legitimate DNS Server MAC	02:00:00:00:00:01
DNS Source Port	54817
DNS Transaction ID	32337
DNS Query	bank.test — A record
Legitimate DNS Response	bank.test → 10.203.0.1
ARP Mapping	10.203.0.1 is-at 02:00:00:00:00:01
ARP Observation	Victim and DNS server exchanged ARP requests/replies confirming the legitimate mapping

DNS Baseline Traffic Analysis

A live DNS traffic capture was performed on the isolated laboratory bridge using tcpdump to establish the legitimate DNS communication pattern before introducing the controlled spoofing scenario. The capture recorded six DNS packets, with no packets dropped by the capture process. The traffic showed the victim system (10.203.0.2) sending DNS A-record queries for bank.test to the legitimate DNS resolver at 10.203.0.1 over UDP port 53. In the most recent successful exchange, the victim used source port 47074 and DNS transaction ID 59721, after which 10.203.0.1 returned an authoritative response resolving bank.test to 10.203.0.1.

for network connectivity. The legitimate DNS baseline demonstrated that the victim's requests for bank.test were resolved by the legitimate DNS endpoint to 10.203.0.1.

The DNS and ARP packet captures provide a preserved reference for the expected communication pattern, while the SHA-256 records provide an integrity mechanism for the collected evidence. The controlled spoofing responder was also prepared within the isolated environment to allow the investigation to examine how an unexpected DNS response could be distinguished from the legitimate resolver through correlation of DNS, ARP, Ethernet, and HTTP evidence.

Overall, the exercise demonstrates that effective DNS-spoofing forensics requires more than examining the returned IP address alone. Establishing a reliable baseline and correlating DNS transactions with source IP addresses, Ethernet MAC addresses, ARP relationships, and subsequent network connections provides stronger evidence for determining whether an observed DNS response represents legitimate variation or controlled anomalous activity. The evidence and supporting scripts have been preserved within the laboratory project structure for further analysis and final verification.